AT-4 ASSIGNMENT

A.Sai Joshitha(192424220)

Deep Learning – CSA4707

1. DEEP FEEDFORWARD NETWORKS FOR CREDIT CARD FRAUD DETECTION

Aim

To design and implement a Deep Feedforward Neural Network (DNN) to detect fraudulent credit card transactions and evaluate its performance using accuracy, precision, recall, and F1-score.

Introduction

A Deep Feedforward Neural Network is an artificial neural network in which information moves from the input layer through one or more hidden layers to the output layer. The information moves forward without forming cycles or feedback loops.

Credit card fraud occurs when someone uses another person's card or card information to perform unauthorized transactions. Traditional fraud detection methods may fail to identify complicated fraud patterns. Deep learning can learn patterns from transaction data and help identify suspicious activities.

The network receives transaction details as input and predicts whether a transaction is genuine or fraudulent. The output is classified into two categories:

- Class 0: Genuine transaction.
- Class 1: Fraudulent transaction.

Objectives

1. To understand the structure of a Deep Feedforward Neural Network.
2. To prepare and preprocess credit card transaction data.
3. To implement a DNN using Python and TensorFlow.

4. To train the network using labelled transaction data.
5. To evaluate the model using accuracy, precision, recall, and F1-score.
6. To understand the advantages and limitations of deep learning in fraud detection.

Problem Statement

Financial institutions process thousands or millions of credit card transactions every day. Identifying fraudulent transactions manually is difficult and time-consuming. A fraud detection system must identify suspicious transactions accurately while minimizing false alarms and missed fraud cases.

The proposed system uses a Deep Feedforward Neural Network to learn from previously labelled transactions and classify new transactions as genuine or fraudulent.

Explanation:

A feedforward neural network consists of three main types of layers.

Input layer: Receives transaction features such as transaction amount and other available numerical transaction attributes.

Hidden layers: Process the input information and learn complex patterns. ReLU activation is commonly used in hidden layers.

Output layer: Produces the probability that a transaction is fraudulent. A sigmoid activation is suitable for binary classification.n

The output of a neuron is calculated as:

$$[\, \boldsymbol{z} = \sum_{\{\boldsymbol{i}=\boldsymbol{1}\}_{\boldsymbol{i}}^{\{\boldsymbol{n}\}}} \boldsymbol{w}\boldsymbol{x}_{\boldsymbol{i}} + \boldsymbol{b} \,]$$

$$[\, \boldsymbol{a} = \boldsymbol{f}(\boldsymbol{z})]$$

Where:

- ($x_{i)}$ = input feature.
- ($w_{i)}$= weight associated with the input.

- (b) = bias.
- (f(z)) = activation function.
- (a) = output of the neuron.

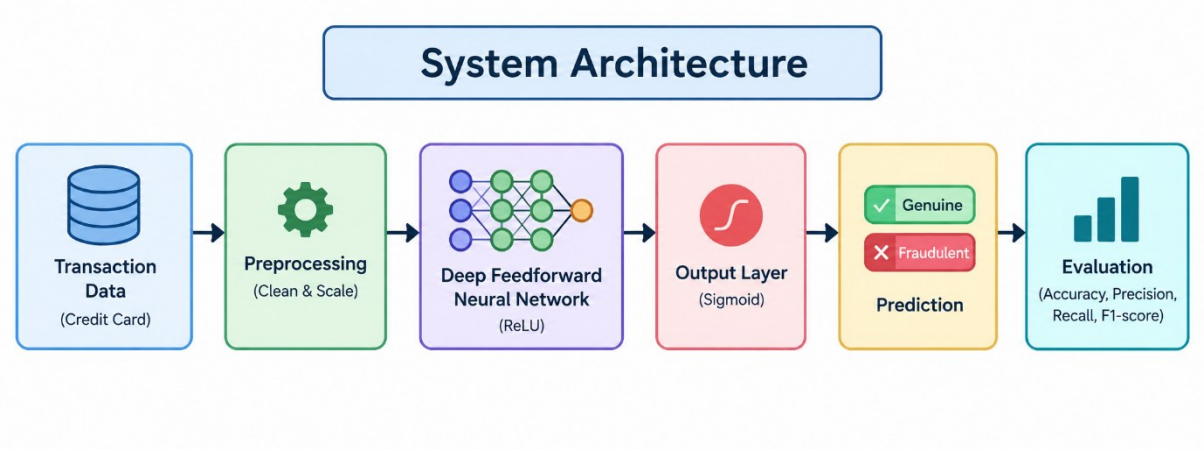
The ReLU activation function is:

[f(z)=max(0,z)]

The sigmoid function is:

[sigma(z)=frac{1}{1+e^{-z}}]

The sigmoid output ranges from 0 to 1 and can be interpreted as the predicted probability of fraction.

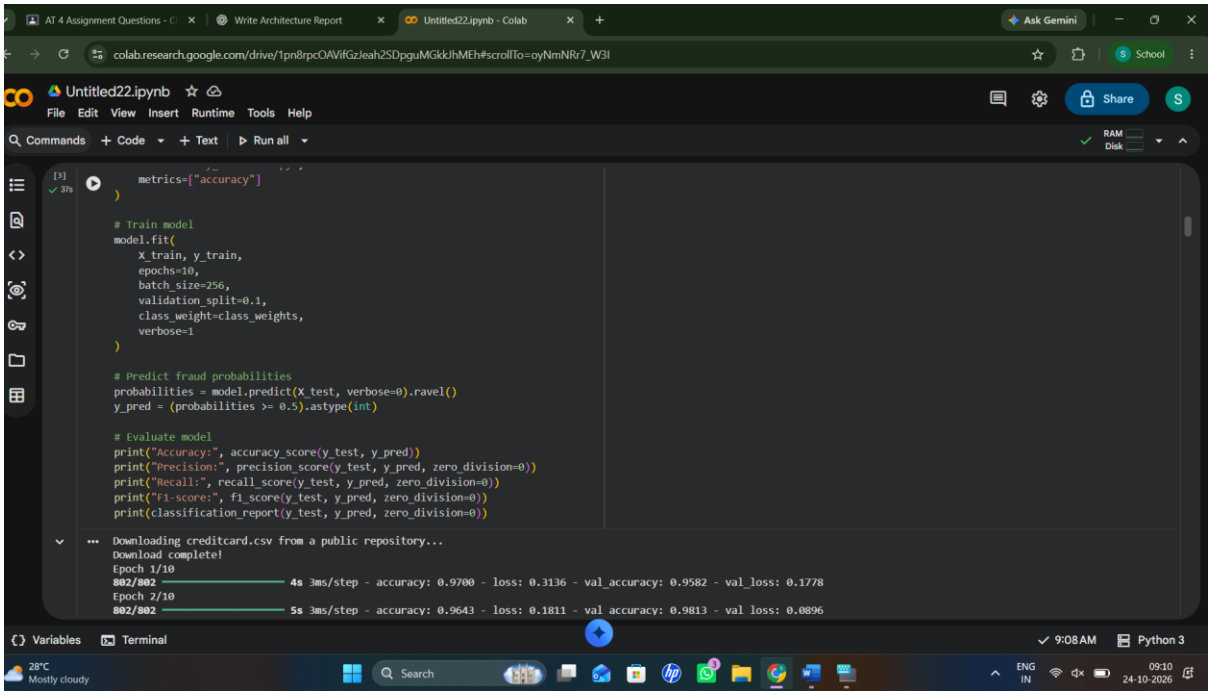


Algorithm

1. Import the required Python libraries.
2. Load the credit card transaction dataset.
3. Separate the input features and target class.
4. Divide the data into training and testing sets.
5. Standardize the numerical features.
6. Build the Deep Feedforward Neural Network.
7. Compile the model using the Adam optimizer and binary cross-entropy loss.
8. Train the model using the training dataset.

9. Predict the class of transactions in the testing dataset.
10. Calculate accuracy, precision, recall, and F1-score.
11. Analyze the model's performance.

Python Implementation



Explanation of the Program

- pandas loads and processes the transaction dataset.
- train_test_split() divides the dataset into training and testing sets.
- StandardScaler() standardizes numerical input features.
- class_weights gives additional importance to the minority class during training.
- Dense(64, activation="relu") creates the first hidden layer with 64 neurons.
- Dropout(0.2) randomly disables some neurons during training to reduce overfitting.
- Dense(32, activation="relu") creates the second hidden layer.
- Dense(1, activation="sigmoid") produces a fraud probability.
- binary_crossentropy measures the difference between predicted probabilities and actual binary labels.

- model.fit() trains the network.
- The final section calculates the evaluation metrics.

Performance Evaluation

```
802/802 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9643 - loss: 0.1811 - val_accuracy: 0.9813 - val_loss: 0.0896
Epoch 3/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9708 - loss: 0.1390 - val_accuracy: 0.9789 - val_loss: 0.0778
Epoch 4/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9708 - loss: 0.1294 - val_accuracy: 0.9604 - val_loss: 0.1176
Epoch 5/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9690 - loss: 0.1173 - val_accuracy: 0.9814 - val_loss: 0.0673
Epoch 6/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9725 - loss: 0.1010 - val_accuracy: 0.9817 - val_loss: 0.0657
Epoch 7/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9724 - loss: 0.0996 - val_accuracy: 0.9508 - val_loss: 0.1360
Epoch 8/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9710 - loss: 0.1011 - val_accuracy: 0.9800 - val_loss: 0.0630
Epoch 9/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9752 - loss: 0.0918 - val_accuracy: 0.9885 - val_loss: 0.0467
Epoch 10/10
802/802 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9773 - loss: 0.0841 - val_accuracy: 0.9855 - val_loss: 0.0520
Accuracy: 0.9834802148800955
Precision: 0.08797653958944282
Recall: 0.9183673469387755
F1-score: 0.16057091882247992
              precision    recall  f1-score   support

           0       1.00      0.98      0.99     56864
           1       0.09      0.92      0.16        98

    accuracy                           0.98     56962
   macro avg       0.54      0.95      0.58     56962
weighted avg       1.00      0.98      0.99     56962
```

The formulas are:

[Accuracy=frac{TP+TN}{TP+TN+FP+FN}]

[Precision=frac{TP}{TP+FP}]

[Recall=frac{TP}{TP+FN}]

[F1=2 frac{Precision\times Recall}{Precision+Recall}]

Here, TP means true positives, TN means true negatives, FP means false positives, and FN means false negatives.

Important observation: Credit card datasets often contain far fewer fraudulent transactions than genuine ones. Therefore, a high accuracy score alone does not prove that the model detects fraud effectively. Precision and recall must also be considered.

Advantages

1. Learns complex transaction patterns automatically.
2. Can process large transaction datasets.
3. Identifies suspicious transactions based on learned patterns.

4. Supports fast prediction after training.
5. Can be retrained when new labelled transaction data becomes available.
6. Provides measurable performance through standard evaluation metrics.

Limitations

1. Requires sufficient labelled transaction data.
2. Training can require considerable computing resources.
3. Imbalanced datasets can affect model performance.
4. Fraud patterns may change over time.
5. Incorrect predictions can inconvenience customers or allow fraudulent transactions.
6. The model must be tested carefully before being used in real financial systems.

Future Scope

1. Use real-time transaction monitoring.
2. Combine the DNN with anomaly detection techniques.
3. Retrain the model as fraud patterns change.
4. Use additional contextual information when legally and appropriately available.
5. Tune the classification threshold to balance missed fraud and false alarms.
6. Develop an alert system for suspicious transactions.

Conclusion

The Deep Feedforward Neural Network provides a method for detecting credit card fraud by learning patterns from labelled transaction data. The model uses hidden layers and activation functions to identify relationships among transaction features. Accuracy, precision, recall, and F1-score help evaluate its performance. In fraud detection, recall and precision are especially important because missed fraud and false alarms have different consequences. Therefore, the model should be evaluated carefully before real-world deployment.

2. GRADIENT-BASED LEARNING FOR IMAGE CLASSIFICATION

Aim

To implement gradient-based learning algorithms, including standard Gradient Descent, Momentum, and RMSprop, for image classification and compare their convergence speed and classification accuracy.

Introduction

Gradient-based learning is an important technique used to train deep neural networks. During training, the network predicts an output and compares it with the correct answer. The difference between the predicted output and the actual output is measured using a loss function.

The learning algorithm calculates gradients to determine how the network's weights should change to reduce the loss. The weights are updated repeatedly until the network learns useful patterns from the training data.

In this case study, the Fashion-MNIST dataset is used. It contains grayscale images of clothing items belonging to ten classes, such as shirts, shoes, bags, and trousers.

Three optimization methods are compared:

1. Standard Stochastic Gradient Descent (SGD).
2. SGD with Momentum.
3. RMSprop.

Objectives

1. To understand the concept of gradient-based learning.
2. To implement a neural network for image classification.
3. To study the working of Gradient Descent, Momentum, and RMSprop.
4. To compare training time, loss, and accuracy.
5. To understand the importance of learning rate selection.
6. To identify an appropriate optimizer for image classification.

Problem Statement

Training a neural network involves finding suitable weights and biases that minimize the loss function. Poor optimization can lead to slow convergence, unstable training, or unsatisfactory accuracy.

The objective is to train the same image-classification network using different optimization algorithms and compare their performance under similar experimental conditions.

Theory

Gradient Descent

Gradient Descent updates model parameters in the direction that reduces the loss function.

The weight update equation is:

$$[w_{t+1}=w_t-eta frac{partial L}{partial w_t}]$$

Where:

- (w_t) = current weight.
- (w_{t+1}) = updated weight.
- (eta) = learning rate.
- (L) = loss function.
- (frac{partial L}{partial w_t}) = gradient of the loss with respect to the weight.

The learning rate controls the size of each update. If it is too small, training may be slow. If it is too large, training may become unstable.

Momentum

Momentum uses information from previous gradients to smooth the weight updates.

\[v_t=\beta v_{t-1}+g_t \]

\[w_{t+1}=w_t-\eta v_t \]

Here, \(g_t\) is the current gradient and \(\beta\) controls the influence of previous gradients.

Momentum can help accelerate learning in consistent directions and reduce oscillations

RMSprop

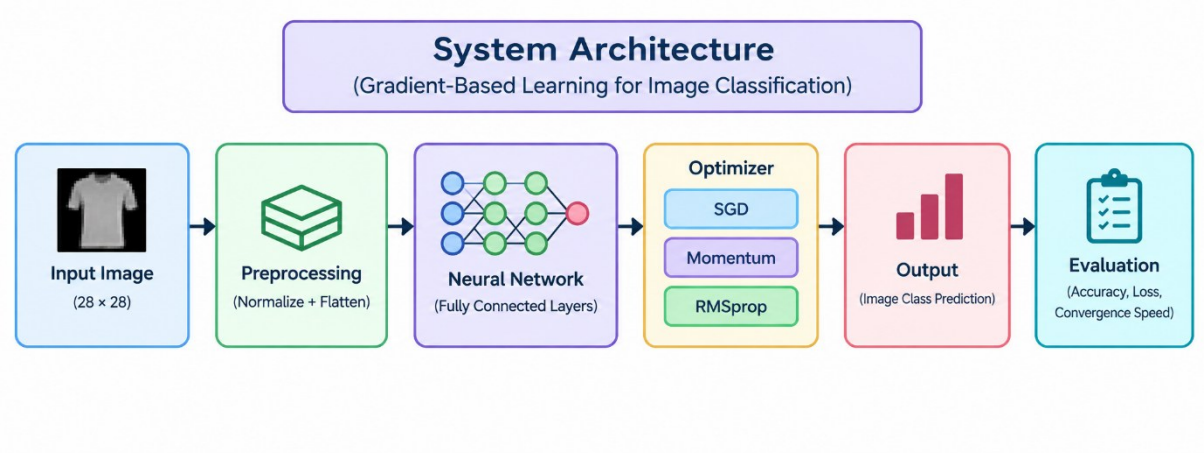
RMSprop maintains an exponentially weighted average of squared gradients.

\[s_t=\rho s_{t-1}+(1-\rho)g_t^2 \]

\[w_{t+1}=w_t-\frac{\eta}{\sqrt{s_t}+\epsilon}g_t \]

Here, \(\rho\) is the decay factor and \(\epsilon\) is a small constant that prevents division by zero.

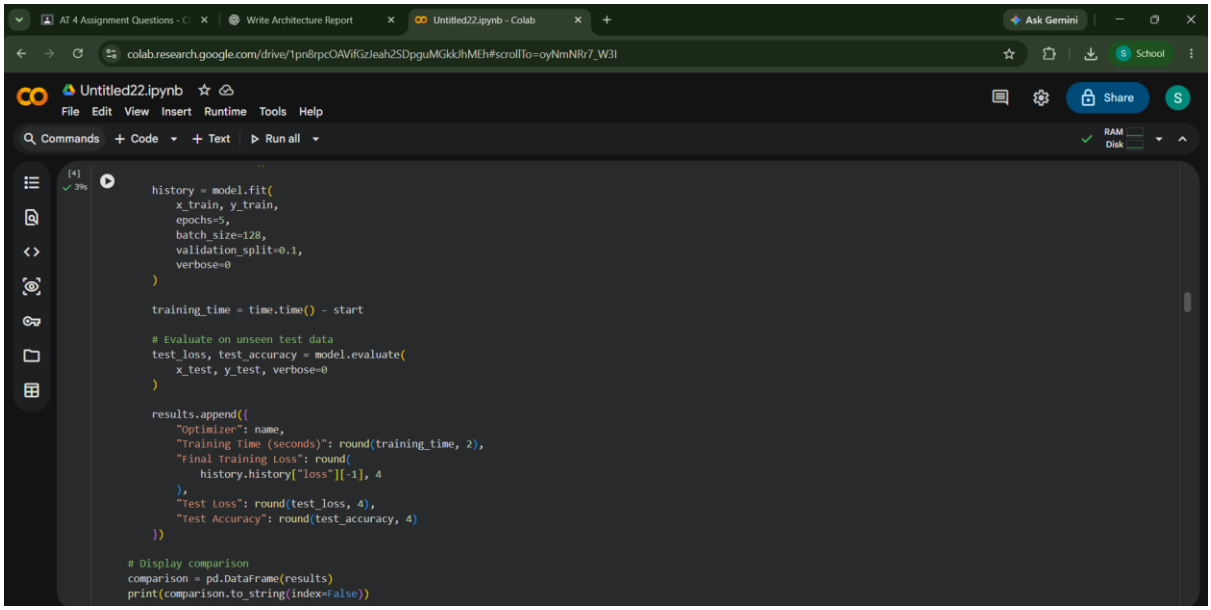
RMSprop adjusts the effective step size for different parameters, which can improve training stability.



Algorithm

1. Import TensorFlow and the required libraries.
2. Load the Fashion-MNIST dataset.
3. Normalize pixel values to the range 0 to 1.
4. Build the neural network.
5. Train the network using SGD.
6. Train the same network using Momentum.
7. Train the same network using RMSprop.
8. Record the training time and test accuracy.
9. Compare the final training losses and validation performance.
10. Identify which optimizer performs best in the experiment.

Python Implementation



Explanation of the Program

- fashion_mnist.load_data() loads training and testing images.
- Dividing pixel values by 255 converts them to a range between 0 and 1.
- Flatten() converts each two-dimensional image into a one-dimensional vector.
- The two hidden layers learn image patterns using ReLU activation.
- softmax produces probabilities for ten clothing categories.
- sparse_categorical_crossentropy is used because the labels are integer class numbers.
- The optimizers dictionary contains SGD, Momentum, and RMSprop.
- time.time() measures the training duration.
- model.evaluate() calculates the loss and accuracy on the test dataset.
- The final table displays the results for comparison.

Performance Comparison

```
        "Optimizer": name,
        "Training Time (seconds)": round(training_time, 2),
        "Final Training Loss": round(
            history.history["loss"][-1], 4
        ),
        "Test Loss": round(test_loss, 4),
        "Test Accuracy": round(test_accuracy, 4)
    })

# Display comparison
comparison = pd.DataFrame(results)
print(comparison.to_string(index=False))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 1us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Optimizer  Training Time (seconds)  Final Training Loss  Test Loss  Test Accuracy
      SGD                   11.82               0.5046     0.5186         0.8184
 Momentum                   10.38               0.3538     0.3821         0.8672
  RMSprop                   11.87               0.3123     0.3548         0.8739
```

SGD: It is simple and computationally inexpensive per update, but it may require more training to reach a good solution.

Momentum: It uses previous gradients to smooth the updates. It may converge faster than standard SGD in some situations.

RMSprop: It adapts the effective learning rate using the history of squared gradients. This can improve stability when parameters have gradients of different magnitudes.

The best optimizer depends on the dataset, network, learning rate, batch size, and number of epochs. The experiment should determine which method performs best under the chosen conditions.

Learning Rate Tuning

The learning rate is one of the most important hyperparameters.

- A very small learning rate may make training slow.
- A very large learning rate may cause the loss to fluctuate or increase.
- A suitable learning rate can help the model learn efficiently.

The learning rate can be tuned by trying several values, such as 0.1, 0.01, and 0.001, while monitoring validation loss and accuracy. The best value should be selected based on validation performance, rather than test accuracy.

Advantages

1. Helps neural networks learn from training examples.
2. Reduces prediction errors by updating model parameters.
3. Momentum can reduce oscillations during optimization.
4. RMSprop adapts update sizes based on gradient history.
5. Performance can be compared using measurable metrics.
6. Optimizers can be applied to many machine-learning tasks.

Limitations

1. Poor learning-rate selection can reduce performance.
2. Training results can vary because of random initialization.
3. Some optimizers may require more computation or memory.
4. A short training run may not show the final convergence behaviour.
5. Results depend on the dataset and model architecture.
6. High training accuracy does not necessarily guarantee good generalization

Future Scope

1. Compare Adam and AdamW with the selected optimizers.
2. Experiment with learning-rate schedules.
3. Increase the number of training epochs.
4. Compare different batch sizes.
5. Test the algorithms on more complex image datasets.
6. Plot training loss and validation accuracy against epochs.

Conclusion

This case study demonstrates how gradient-based optimization algorithms train neural networks for image classification. Standard SGD, Momentum, and RMSprop update model parameters differently. Comparing their training time, loss, and test accuracy helps determine

their suitability for a particular task. Learning rate, batch size, network architecture, and dataset characteristics must also be considered when selecting an optimizer.

3. HIDDEN UNITS AND ACTIVATION FUNCTIONS FOR SPEECH RECOGNITION

Aim

To design and evaluate neural networks with different numbers of hidden units and activation functions for speech recognition, and to analyze how these choices affect classification accuracy, training time, and generalization.

Introduction

Speech recognition is the process of converting spoken audio into meaningful information, such as words or sound categories. It is used in voice assistants, automated customer services, speech-to-text applications, and voice-controlled devices.

A neural network processes numerical representations of audio rather than directly understanding sound in the same way humans do. Speech signals are often converted into features such as Mel-frequency cepstral coefficients (MFCCs) or log-Mel spectrograms.

Hidden units learn relationships between these features. The number of hidden units affects the network's learning capacity, while the activation function determines how neurons transform their inputs.

This case study compares different hidden-layer sizes and the ReLU, sigmoid, and tanh activation functions.

Objectives

1. To understand the role of hidden units in a neural network.
2. To understand activation functions used in deep learning.
3. To prepare numerical features extracted from speech signals.
4. To compare networks with different numbers of hidden units.
5. To evaluate ReLU, sigmoid, and tanh activation functions.
6. To recommend a suitable network configuration for speech classification.

Problem Statement

Speech signals contain complex patterns that vary with pronunciation, speaker, background noise, and recording conditions. A network with too few hidden units may fail to learn important patterns, while a very large network may require more resources and overfit the training data.

The goal is to identify a suitable number of hidden units and activation function by comparing models on the same labelled speech dataset.

Theory

Hidden Units

A hidden unit receives inputs, applies weights and bias, and produces an output through an activation function.

$$z=\sum_{i=1}^{n}w_i x_i+b$$

$$a=f(z)$$

A network with more hidden units can learn more complicated relationships, but it may require more data and computation.

ReLU Activation

$$ReLU(x)=\max(0,x)$$

ReLU outputs zero for negative inputs and returns the input for positive values. It is widely used in hidden layers because it is simple to calculate.

Sigmoid Activation

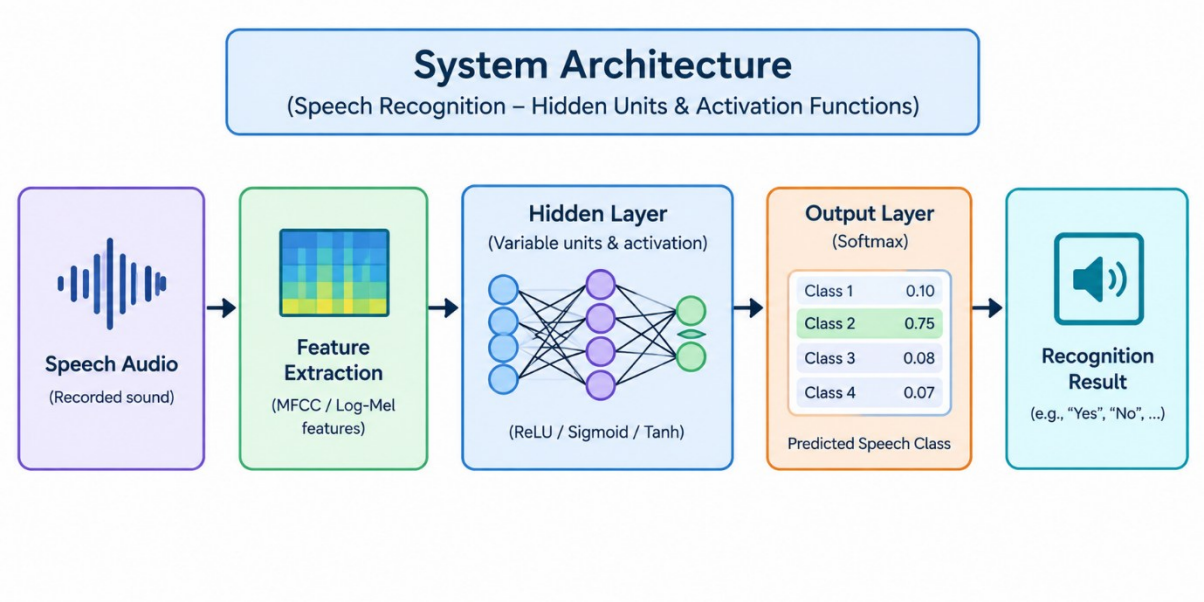
$$\sigma(x)=\frac{1}{1+e^{-x}}$$

Sigmoid produces outputs between 0 and 1. It can be useful for binary output probabilities, but its gradients can become very small for large positive or negative inputs.

Tanh Activation

$$\tanh(x)=\frac{e^x-e^{-x}}{e^x+e^{-x}}$$

Tanh produces values between -1 and 1. Its outputs are centred around zero, but its gradients can also become small when the input has a large magnitude.



Dataset Preparation

Used a labelled speech dataset in which each audio sample has a class label. For a simple experiment, each recording can be converted into a fixed-length numerical feature vector containing MFCC features.

prepared a file named speech_features.csv with the following structure:

- Each row represents one speech recording.
- The numerical columns contain extracted speech features.
- The final column is named label.
- Each label identifies a word, command, or speech category.

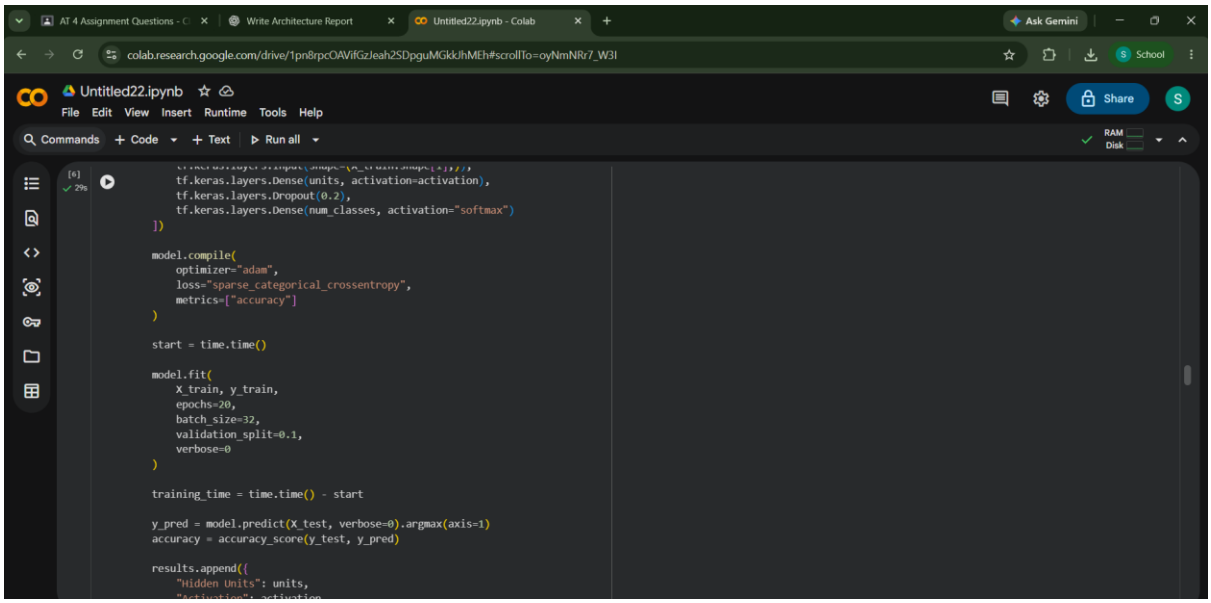
For example, the labels may represent words such as “yes,” “no,” “up,” and “down.” These are examples only; the labels must match the dataset being used.

Algorithm

1. Collect a labelled speech dataset.
2. Extract MFCC or log-Mel features from each audio recording.
3. Arrange the features and labels in a CSV file.
4. Load the data and encode the class labels.
5. Divide the data into training and testing sets.

6. Standardize the feature values.
7. Build networks with different hidden-unit counts and activation functions.
8. Train each network under the same conditions.
9. Measure test accuracy and training time.
10. Compare the results and recommend a suitable configuration.

Python Implementation



Explanation of the Program

- speech_features.csv contains the extracted features and corresponding labels.
- LabelEncoder() converts text labels into numerical class IDs.
- train_test_split() divides the data into training and testing sets.
- StandardScaler() standardizes the feature values.
- The experiments list specifies the hidden-unit counts and activation functions to compare.
- Dense(units, activation=activation) creates a hidden layer with the selected configuration.
- Dropout(0.2) helps reduce overfitting during training.
- The softmax output layer predicts probabilities for all speech classes.

- model.fit() trains each network.
- accuracy_score() measures the percentage of correct predictions on the test set.
- The results table compares the different configurations.

Performance Evaluation

```
            "Activation": activation,
            "Test Accuracy": round(accuracy, 4),
            "Training Time": round(training_time, 2)
        })

    print(pd.DataFrame(results).to_string(index=False))
```

```
Downloading speech_features.csv from public repository...
Download complete!
 Hidden Units Activation  Test Accuracy  Training Time
           16       relu         0.9748           6.51
           64       relu         0.9795           5.21
          128       relu         0.9811           5.79
           64    sigmoid         0.9716           5.87
           64       tanh         0.9763           5.25
```

16 hidden units: This network is relatively small and may train quickly. However, it may not have enough capacity to learn complicated speech patterns.

64 hidden units: This configuration provides greater learning capacity than the smaller network while keeping the model relatively compact.

128 hidden units: This network can learn more complex relationships, but it may require more computation and can overfit if the dataset is small.

ReLU: Often a useful starting point for hidden layers because it is computationally simple.

Sigmoid: Can experience small gradients in saturated regions, which may make learning difficult in deeper networks.

Tanh: Produces zero-centred activations, but can also suffer from small gradients in saturated regions.

The best configuration must be selected from actual validation and test results. A larger network or a particular activation function is not guaranteed to perform best on every speech dataset.

Advantages

1. Hidden units allow networks to learn complex speech patterns.
2. Different activation functions provide different nonlinear transformations.
3. MFCC features provide a compact numerical representation of audio.
4. The model can classify speech commands or word categories.
5. Performance can be measured using accuracy and training time.
6. The experiment supports systematic comparison of network designs.

Limitations

1. Speech recordings may contain background noise.
2. Different speakers may pronounce the same word differently.
3. Poor feature extraction can reduce classification accuracy.
4. A large number of hidden units may increase training time.
5. Small datasets can lead to overfitting.
6. A simple feedforward network may not capture all temporal relationships in speech.

Future Scope

1. Use larger speech datasets containing different speakers.
2. Compare MFCC features with log-Mel spectrograms.
3. Implement convolutional neural networks for speech features.
4. Explore recurrent neural networks and Transformer-based speech models.
5. Apply noise reduction and data augmentation.
6. Evaluate models using precision, recall, and a confusion matrix in addition to accuracy.

Conclusion

This case study explains how hidden-unit count and activation functions influence neural network performance in speech classification. Small networks may train quickly but may lack sufficient learning capacity. Larger networks can model more complex patterns but may need

more data and computation. ReLU, sigmoid, and tanh have different mathematical properties that influence gradient flow and learning. The most suitable configuration should be selected through experiments on genuine speech data.

4.ARCHITECTURE DESIGN FOR AUTONOMOUS DRIVING

Aim

To design and compare shallow, deep, and wide neural network architectures for an autonomous-driving classification task and evaluate their accuracy, computation time, and generalization ability.

Introduction

Autonomous driving is a technology that allows vehicles to perceive their surroundings and make driving decisions with limited or no direct human control. It uses information from cameras, radar, lidar, GPS, and other vehicle sensors to understand the road environment.

Deep learning can identify objects, detect lanes, classify road conditions, and support driving decisions. However, the architecture of a neural network affects its ability to learn, its computational requirements, and how well it performs on unseen data.

A shallow network has fewer hidden layers. A deep network has several hidden layers. A wide network has more neurons in each hidden layer.

This case study compares these architectures using labelled driving-related sensor features. The experiment demonstrates architecture comparison; it is not a complete autonomous-driving control system.

Objectives

1. To understand neural network architecture design.
2. To compare shallow, deep, and wide networks.
3. To study the effect of architecture on classification accuracy.
4. To measure training time and model size.
5. To examine overfitting and generalization.
6. To recommend an architecture for a driving-related classification task.

Problem Statement

An autonomous vehicle must make reliable decisions based on information about its surroundings. A model that is too simple may fail to learn complex driving patterns, while an excessively large model may require more computing power and may overfit the training data.

The objective is to compare different neural network architectures and identify a suitable balance between predictive performance, computational cost, and generalization.

Theory

Shallow Neural Network

A shallow network contains one or a small number of hidden layers.

Advantages:

- Simple to implement.
- Requires fewer computations.
- May train quickly on smaller datasets.

Limitations:

- May struggle to represent complicated patterns.
- May have limited learning capacity for complex tasks.

Deep Neural Network

A deep network contains multiple hidden layers. Each layer can learn different representations of the input.

For example, in image processing, early layers may learn basic visual patterns, while later layers can combine these into more complex representations.

Advantages:

- Can learn hierarchical features.
- Can model complex relationships.
- Suitable for many difficult classification tasks.

Limitations:

- May require more training time and data.
- Can be more difficult to optimize.
- May overfit without suitable regularization.

Wide Neural Network

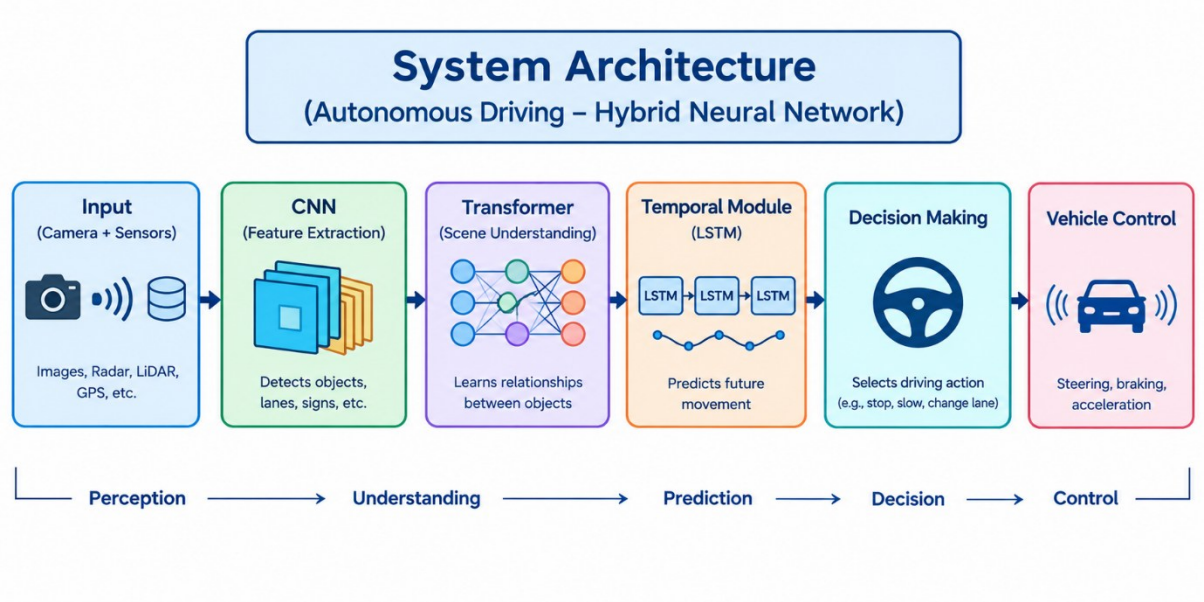
A wide network contains more neurons within its hidden layers.

Advantages:

- Can increase learning capacity.
- May perform well on complex feature relationships.

Limitations:

- Requires additional memory and computation.
- May overfit when training data is limited.
- More neurons do not always produce better results.



Dataset Preparation

Prepared a labelled dataset named driving_data.csv. Each row should represent one observation, and the dataset should contain numerical driving-related features and a target column named label.

Example input features may include:

- Vehicle speed.
- Distance from the lane centre.
- Distance to the nearest obstacle.
- Road curvature.
- Relative speed of a nearby object.

The label may represent a task such as classifying a driving state into categories such as normal driving, caution, or braking. These are illustrative categories; the dataset must contain the labels used in the experiment.

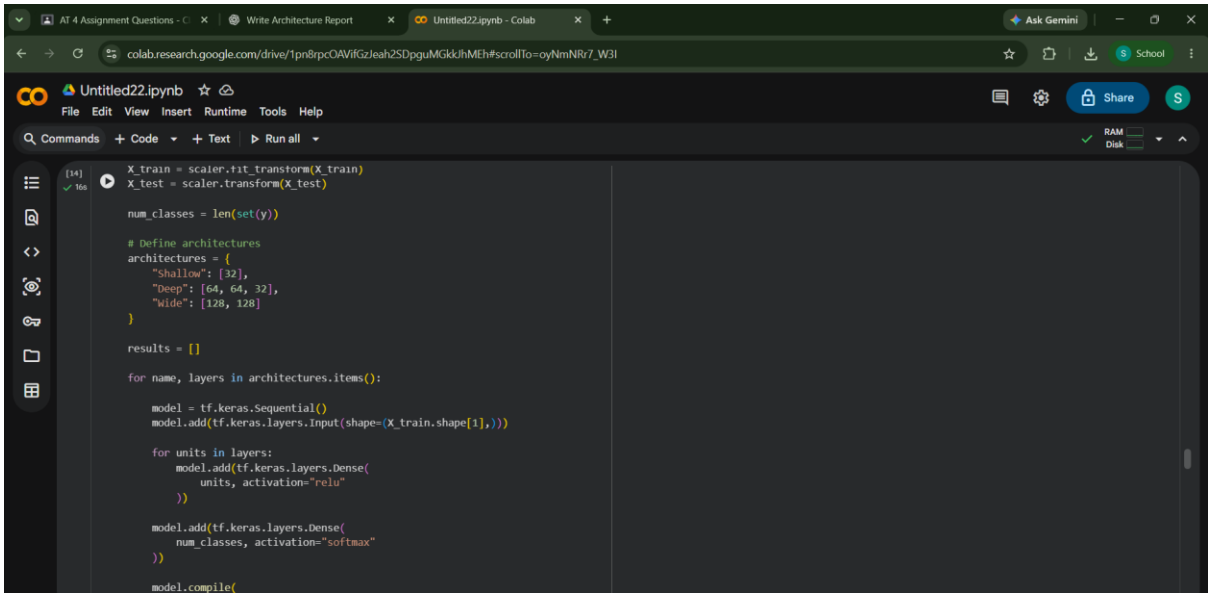
The model should only be trained on features that would be available at prediction time. Training and testing data should also be separated appropriately to prevent data leakage.

Algorithm

1. Import the required libraries.
2. Load the labelled driving dataset.
3. Separate the features and target labels.
4. Split the data into training and testing sets.
5. Standardize the numerical features.
6. Build a shallow network.
7. Build a deep network.
8. Build a wide network.
9. Train all models using the same training conditions.
10. Measure test accuracy, test loss, training time, and number of parameters.

11. Compare the results and select an appropriate architecture.

Python Implementation



Explanation of the Program

- driving_data.csv contains labelled driving-related observations.
- train_test_split() separates training data from test data.
- StandardScaler() puts numerical features on comparable scales.
- architectures defines the three networks being compared.
- [32] represents one hidden layer containing 32 neurons.
- [64, 64, 32] represents a deeper network with three hidden layers.
- [128, 128] represents a wide network with two hidden layers of 128 neurons each.
- ReLU is used in the hidden layers.
- Softmax produces class probabilities at the output.
- model.count_params() measures the number of trainable and non-trainable model parameters.
- time.time() measures the duration of training.
- model.evaluate() measures test loss and accuracy.

Performance Evaluation

```
            "Training Time": round(training_time, 2),
            "Test Loss": round(test_loss, 4),
            "Test Accuracy": round(test_accuracy, 4)
        })

    print(pd.DataFrame(results).to_string(index=False))
```

```
Generating local synthetic driving_data.csv...
Generation complete!
Architecture  Parameters  Training Time  Test Loss  Test Accuracy
     Shallow         836           7.07     0.3805         0.7832
        Deep        7780           4.64     0.4458         0.7803
        Wide       19844           4.46     0.4483         0.7601
```

Shallow network: It may be a suitable choice when the dataset is relatively simple and low computational cost is important.

Deep network: It may learn more complex relationships by combining features across several layers. However, it can require more training and careful regularization.

Wide network: It may learn many feature relationships because it contains more neurons, but its larger size can increase computation and memory requirements.

The architecture with the highest training accuracy is not necessarily the best model. Test performance and the difference between training and validation performance should be considered to identify overfitting.

Architecture Selection for Autonomous Driving

The best architecture depends on the driving task.

- For simple sensor-based classification, a small feedforward network may be sufficient.
- For camera-based object detection, convolutional neural networks or vision Transformer architectures may be more suitable.
- For combining camera, radar, and other sensor information, a sensor-fusion architecture can be used.
- For analysing information over time, a temporal model can help process sequences of observations.

Real autonomous-driving systems also require reliable perception, safety checks, testing in different environments, and a control system. The simple network comparison above is only one part of the overall process.

Advantages

1. Provides a systematic method to compare network structures.
2. Helps balance accuracy and computational cost.
3. Demonstrates the effect of depth and width.
4. Supports selection of a suitable architecture for the available data.
5. Helps identify possible overfitting.
6. Can be extended to more complex sensor and image datasets.

Limitations

1. The simple feedforward model does not process raw camera images directly.
2. Results depend on the quality and diversity of the dataset.
3. Larger architectures may require more computing resources.
4. High classification accuracy does not guarantee safe vehicle operation.
5. A limited test dataset may not represent every road condition.
6. A real vehicle requires additional safety and control mechanisms.

Future Scope

1. Use real-world autonomous-driving datasets.
2. Implement CNN-based object detection.
3. Combine information from multiple sensors.
4. Evaluate performance under rain, fog, and poor lighting.
5. Use temporal models for motion prediction.
6. Test the system in a driving simulator before real-world use.

Conclusion

This case study compares shallow, deep, and wide neural network architectures for a driving-related classification task. A shallow network may be computationally efficient, while deeper or wider networks may learn more complex relationships. The most appropriate design must

be selected based on test accuracy, training time, model size, and generalization. In autonomous driving, model performance must also be combined with extensive safety testing before any real-world use.

5. BACKPROPAGATION AND DIFFERENTIATION FOR A RECOMMENDATION SYSTEM

Aim

To implement the backpropagation algorithm for a neural-network-based recommendation system, compare activation functions, and study methods for identifying and reducing vanishing and exploding gradients.

Introduction

A recommendation system suggests products, movies, songs, courses, or other items based on user preferences and available information. For example, a movie recommendation system can learn patterns from user ratings and recommend movies that a user may enjoy.

Deep learning can model relationships between users, items, and their features. During training, the network predicts a rating or preference score and compares it with the actual rating. The error is then propagated backward through the network to update its weights.

This process is called backpropagation. It uses differentiation and the chain rule to calculate gradients.

However, gradients can become extremely small or excessively large in some networks. These problems are known as the vanishing gradient and exploding gradient problems.

Objectives

1. To understand backpropagation in neural networks.
2. To implement forward propagation and backward propagation.
3. To use a neural network for predicting user ratings.
4. To compare sigmoid, tanh, ReLU, and Leaky ReLU activation functions.
5. To understand vanishing and exploding gradients.
6. To study techniques for improving gradient flow and training stability.

Problem Statement

Users may have different preferences, and the same item may receive different ratings from different users. A recommendation system must learn relationships between input features and user ratings.

Training the network requires calculating errors and adjusting weights. If gradients become too small, earlier layers may learn very slowly. If gradients become too large, weight updates may become unstable.

The objective is to implement backpropagation and investigate how activation functions and gradient-management techniques influence the training process.

Theory

Forward Propagation

Forward propagation sends input information through the network to generate a prediction.

For a hidden layer:

$$z_1=W_1x+b_1$$

$$a_1=f(z_1)$$

For the output layer:

$$z_2=W_2a_1+b_2$$

$$\hat{y}=g(z_2)$$

Where:

- x = input features.
- W_1,W_2 = weight matrices.
- b_1,b_2 = bias values.
- f = hidden-layer activation function.
- g = output activation function.
- $\hat{y}$ = predicted rating.

Loss Function

The loss function measures the difference between the predicted rating and the actual rating.

For regression tasks, Mean Squared Error (MSE) is commonly used:

$$\mathbf{MSE=\frac{1}{N}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2}$$

Where:

- y_i = actual rating.
- $\hat{y}_i$ = predicted rating.
- N = number of examples.

A smaller MSE indicates that the predictions are closer to the actual ratings.

Backpropagation

Backpropagation calculates gradients of the loss with respect to the network's parameters. These gradients are used to update weights and biases.

The general weight-update equation is:

$$W_{\text{new}}=W_{\text{old}}-\eta\frac{\partial L}{\partial W}$$

Here, η is the learning rate and $\frac{\partial L}{\partial W}$ is the gradient of the loss with respect to the weights.

The chain rule is used to calculate derivatives across multiple layers. This allows the error at the output to influence weight updates in earlier layers.

Activation Functions

Sigmoid:

$$\mathbf{\sigma(x)=\frac{1}{1+e^{-x}}}$$

It produces outputs between 0 and 1. Its gradient becomes small when the input is very positive or very negative.

Tanh:

$$\mathbf{\tanh(x)=\frac{e^x-e^{-x}}{e^x+e^{-x}}}$$

It produces outputs between -1 and 1. It is zero-centred, but its gradient can become small for large input magnitudes.

ReLU:

$$ReLU(x)=\max(0,x)$$

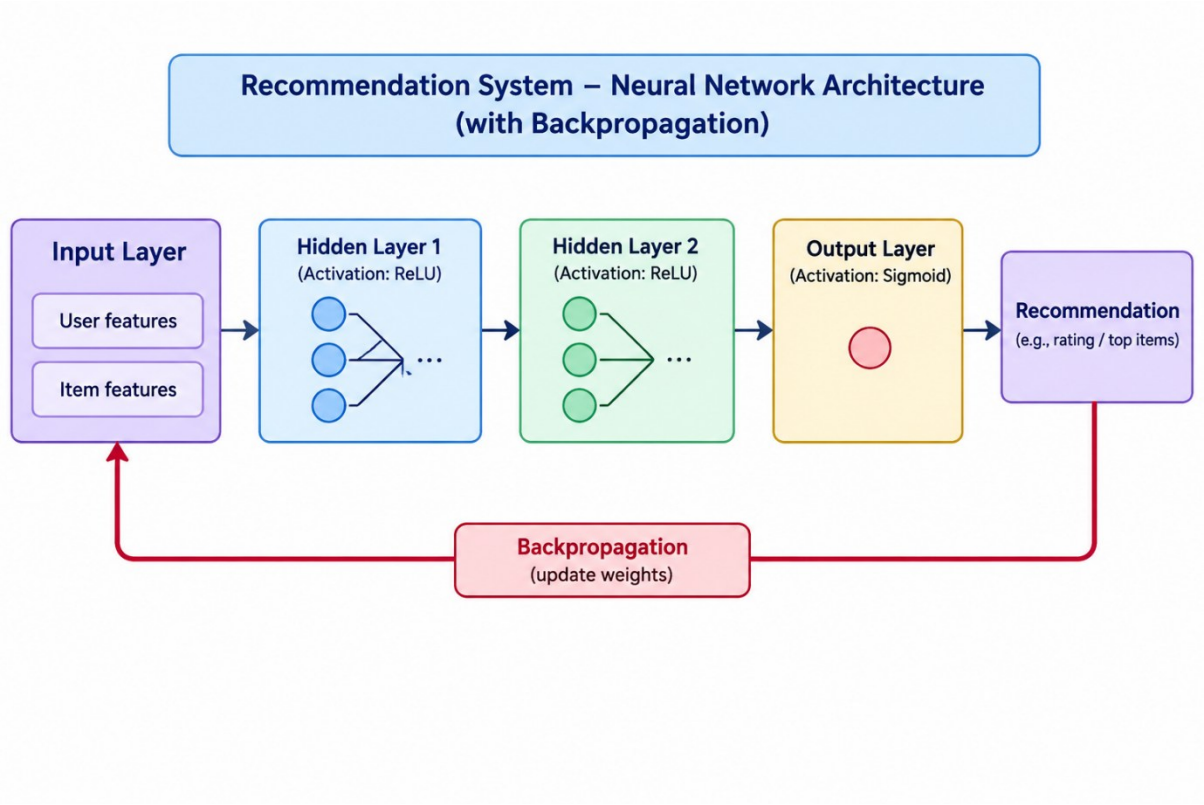
It returns zero for negative inputs and the input itself for positive inputs. It is commonly used in hidden layers, although inactive neurons can sometimes stop receiving useful gradients.

Leaky ReLU:

$$f(x)= \begin{cases} x,&x>0\\ 0.01x,&x\leq0 \end{cases}$$

It allows a small non-zero output for negative inputs, which can help reduce the problem of neurons becoming permanently inactive.

System Architecture

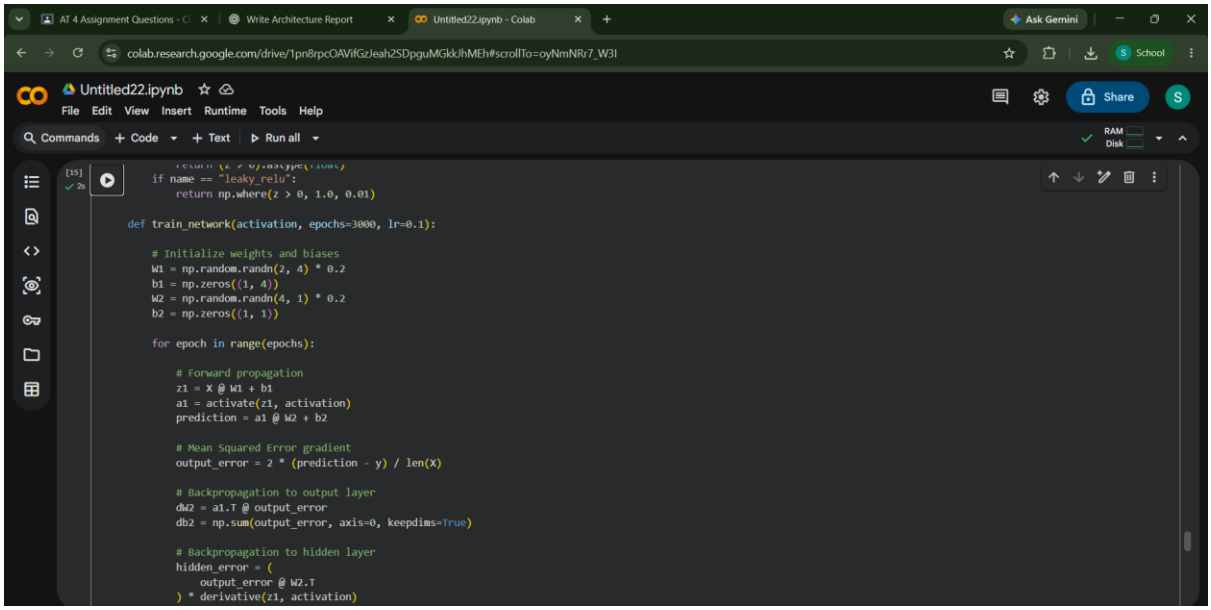


Algorithm

1. Create a labelled user-rating dataset.
2. Normalize the input features.
3. Initialize the network weights and biases.
4. Perform forward propagation to predict a rating.

5. Calculate the loss using MSE.
6. Calculate gradients using the chain rule.
7. Propagate gradients backward through the network.
8. Update the weights and biases using gradient descent.
9. Repeat the process for multiple epochs.
10. Compare the activation functions.
11. Analyze gradient behaviour and prediction error.
12. Identify methods to improve training stability.

Python Implementation



Explanation of the Program

- X contains two numerical features for each illustrative user-item example.
- y contains the corresponding normalized target ratings.
- activate() implements four activation functions.
- derivative() calculates the derivative needed during backpropagation.
- W1 and b1 represent the hidden-layer weights and biases.
- W2 and b2 represent the output-layer weights and bias.

- X @ W1 + b1 calculates the weighted input to the hidden layer.
- a1 @ W2 + b2 calculates the predicted rating.
- The MSE gradient measures how the prediction error changes with respect to the output.
- dW2 and db2 calculate gradients for the output layer.
- hidden_error propagates the error back through the hidden layer using the activation derivative.
- dW1 and db1 calculate gradients for the hidden layer.
- The weight-update statements adjust the parameters in the direction that reduces the loss.
- The final section prints the MSE for each activation function.

Because this is a small illustrative dataset, the final MSE values should not be treated as evidence of real-world recommendation accuracy.

Performance Evaluation

```
        print(np.round(predictions.flatten(), 3))

...
    Activation: sigmoid
    Final MSE: 0.002784
    Predicted ratings:
    [0.177 0.649 0.407 0.787 0.416 0.731 0.425 0.739 0.644 0.924]

    Activation: tanh
    Final MSE: 0.002826
    Predicted ratings:
    [0.179 0.649 0.408 0.787 0.416 0.731 0.423 0.738 0.64  0.928]

    Activation: relu
    Final MSE: 0.0489
    Predicted ratings:
    [0.59 0.59 0.59 0.59 0.59 0.59 0.59 0.59 0.59 0.59]

    Activation: leaky_relu
    Final MSE: 0.002964
    Predicted ratings:
    [0.194 0.644 0.407 0.788 0.413 0.726 0.419 0.732 0.632 0.945]
```

Sigmoid: Its output is bounded between 0 and 1. For large positive or negative inputs, the derivative becomes small, which can slow learning in networks with many layers.

Tanh: Its outputs are centred around zero, but the derivative also becomes small when its input magnitude is large.

ReLU: It allows gradients to pass through positive activations, but neurons that remain inactive may receive zero gradients.

Leaky ReLU: It retains a small gradient for negative inputs, which can help prevent neurons from becoming completely inactive.

The activation function with the lowest MSE in this particular experiment has the smallest training error. However, the best recommendation model must also be tested on unseen user-item examples.

Vanishing and Exploding Gradients

Vanishing Gradient Problem

The vanishing gradient problem occurs when gradients become extremely small as they are propagated backward through a network.

Causes:

1. Repeated multiplication by small derivatives.
2. Saturated sigmoid or tanh activations.
3. Poor weight initialization.
4. Very deep network architectures.

Effects:

1. Earlier layers learn very slowly.
2. Training may stop improving.
3. The model may fail to learn important patterns.
4. Convergence may become difficult.

Solutions:

- Use ReLU or suitable variants in hidden layers.
- Apply appropriate weight initialization.

- Use normalization techniques when suitable.
- Use residual connections in deeper networks.
- Monitor gradient magnitudes during training.

Exploding Gradient Problem

The exploding gradient problem occurs when gradients become excessively large during backpropagation.

Causes:

1. Repeated multiplication by large derivatives.
2. Poor weight initialization.
3. Excessively large learning rates.
4. Unstable training dynamics.

Effects:

1. Weight updates become excessively large.
2. Loss may fluctuate or increase.
3. Training can become unstable.
4. Numerical overflow may occur.

Solutions:

- Reduce the learning rate.
- Use gradient clipping.
- Apply suitable weight initialization.
- Monitor training loss and gradient magnitudes.
- Use appropriate normalization methods where necessary.

Applications

1. Movie recommendation systems.
2. Online shopping recommendations.

3. Music and video suggestions.
4. Personalized learning platforms.
5. News and content recommendations.
6. Product ranking systems.

Advantages

1. Backpropagation efficiently calculates gradients for network parameters.
2. Neural networks can learn complex user-item relationships.
3. Different activation functions can be compared experimentally.
4. Gradient analysis helps identify training difficulties.
5. Suitable optimization techniques can improve training stability.
6. The approach can be extended to larger recommendation datasets.

Limitations

1. The illustrative dataset is too small to represent real user behaviour.
2. Users may have incomplete or inconsistent rating histories.
3. New users and new items can create a cold-start problem.
4. Incorrect recommendations can reduce user satisfaction.
5. Deep networks may require substantial training data.
6. Bias in the dataset can affect recommendation quality.

Future Scope

1. Use real datasets such as MovieLens for movie recommendations.
2. Combine user and item embeddings.
3. Compare neural networks with collaborative filtering.
4. Use regularization to reduce overfitting.
5. Apply gradient clipping to control excessive gradient values.

6. Evaluate recommendation quality using RMSE, precision at K, recall at K, and ranking metrics.

Conclusion

This case study explains backpropagation and its use in a neural-network-based recommendation system. Forward propagation produces predicted ratings, while backpropagation calculates gradients and updates the network parameters. Sigmoid, tanh, ReLU, and Leaky ReLU have different effects on gradient flow. Vanishing and exploding gradients can make training difficult, but suitable activation functions, initialization, learning rates, normalization, and gradient clipping can help improve stability. Real-world performance must be evaluated on a suitable user-item dataset.